

CPSC 250

Functions

Edward Brash

1 Introduction

Functions are one of the most important organizational tools in programming.

Functions allow us to:

- divide problems into smaller pieces,
- reuse code,
- improve readability,
- and reduce duplication.

2 Basic Function Structure

General syntax:

```
def my_function(arguments):  
  
    # code here  
  
    return values
```

Important parts:

- Function name
- Argument list
- Function body
- Return value list

2.1 Argument List

The argument list contains information passed *to* the function.

2.2 Return Value List

The return value list contains information returned *from* the function.

3 Conceptual Model

Conceptually:

Main Program $\rightarrow$ Function

Arguments flow into the function.

Return values flow back out.

4 Example: add() Function

```
def add(a, b):  
    c = a + b  
    return c
```

Example usage:

```
x = 4  
y = 6  
  
z = add(x, y)  
  
print(z)
```

Output:

```
10
```

5 Very Important Idea: Pass by Value

Inside the function:

- **a** and **b** are copies of **x** and **y**
- they are different variables in memory

Changing variables inside the function does not change variables in the main program.

This is called:

pass by value

6 Naming Convention Recommendation

A very good habit is to use different variable names inside functions than in the main program.

Example:

Main Program	Function
x, y, z	a, b, c

This reinforces the idea that these are different variables occupying different memory locations.

7 Dynamic Typing in Python

Unlike languages such as:

- C
- C++
- Java
- Go

Python uses dynamic typing (sometimes informally called “duck typing”).
The same function may operate on multiple data types.

Example:

```
def add(a, b):  
    return a + b
```

This works for integers:

```
print(add(4, 6))
```

but also for strings:

```
x = "that_is"  
y = "so_cool!"  
  
z = add(x, y)  
  
print(z)
```

Output:

```
that is so cool!
```

Python automatically determines the correct behavior based on object types.

8 Variable Scope

Scope determines where variables have meaning inside a program.

8.1 Example

```
employee_name = 'Bob'  
  
def get_name():  
    name = input()  
    employee_name = name  
  
get_name()  
  
print(employee_name)
```

Important:
The variable:

```
employee_name
```

inside the function is different from the variable outside the function.
Thus, the printed value remains:

```
Bob
```

unless special mechanisms such as `global` are used.

9 Pass by Reference?

Students often hear the phrase:

“Python passes lists by reference.”

This is not quite correct.

10 The Common Misconception

Suppose we have:

```
numbers = [1, 2, 3, 4, 5]
```

and:

```
def change_numbers(nums):  
    nums[0] = 7
```

Then:

```
change_numbers(numbers)  
print(numbers)
```

produces:

```
[7, 2, 3, 4, 5]
```

Many people incorrectly conclude that Python uses pass-by-reference.

11 What is Actually Happening

The important distinction is:

- the list itself exists as a mutable object in memory,
- the function receives a copy of the reference,
- both variables point to the same underlying object.

The variables themselves are still separate variables.

12 Mutable vs Immutable Objects

This behavior depends heavily on mutability.

12.1 Immutable Objects

Examples:

- integers
- floats
- strings
- tuples

cannot be modified in-place.

12.2 Mutable Objects

Examples:

- lists
- dictionaries
- sets

can be modified in-place.

13 Key Takeaways

- Functions help organize and reuse code.
- Arguments pass information into functions.
- Return values pass information back out.
- Python primarily behaves like pass-by-value.
- Mutable objects can create reference-sharing behavior.
- Scope determines where variables exist and have meaning.
- Python uses dynamic typing.